

Ejercicio 1

Sea el siguiente programa de transferencia en cuentas bancarias:

```
program transfer;
begin
START
    input (cta_origen, cta_destino, monto);
    tmp := read(cta_origen); /* lee saldo en cta_origen */
    if (tmp < monto) {
        output ("saldo insuficiente");
        ABORT
    } else {
        write (cta_origen, tmp-monto); /* actualizo saldo */
        tmp := read (cta_destino); /* leo saldo en cta_destino */
        write (cta_destino, tmp+monto); /* actualizo saldo */
        COMMIT
        output ("transacción completa");
    }
end program.
```

y el siguiente programa que imprime la suma de los saldos de dos cuentas:

```
program suma2ctas;
begin
    input (cta1, cta2);
    tmp1 := read (cta1);
    tmp2 := read (cta2);
    output (tmp1+tmp2);
end program.
```

- a. Dar un ejemplo de una ejecución concurrente de los programas anteriores tal que el resultado de suma2ctas sea incorrecto.
- b. Dar un ejemplo de una ejecución concurrente de dos transferencias
T1: r1(x), w1(x), r1(y), w1(y) C1 y T2: r2(x), w2(x), r2(y), w2(y) C2 tal que:
 1. sea recuperable pero no serializable.
 2. sea serializable pero no recuperable.
 3. no sea serializable ni recuperable.

Ejercicio 2

Sean las transacciones:

T1: w1(x), r1(y), C1

T2: r2(x), w2(y), C2

- a. Dar una historia o explicar porqué no existe una historia (que contenga los COMMIT), que sea recuperable y que presente las operaciones en el siguiente orden:
w1(x), r2(x), w2(y), r1(y)
- b. Dar una ejecución concurrente de T1 y T2 tal que la historia que representa a esa ejecución sea recuperable. Decir si es serializable.

Ejercicio 3

Sean las siguientes transacciones:

T0:

```
read(a);  
read(b);  
if a = 0 then b := b+1;  
write(b).
```

T1:

```
read(b);  
read(a);  
if b = 0 then a := a+1;  
write(a).
```

Sea la siguiente restricción de integridad:

$a = 0$ o $b = 0$, con $a = b = 0$ los valores iniciales.

- Mostrar que toda ejecución serial que involucra a T0 y T1 preserva la consistencia de la base de datos.
- Dar una ejecución concurrente de T0 y T1 que no sea serializable.
- ¿Existe una ejecución concurrente de T0 y T1 que sea serializable?

Ejercicio 4

- Probar que si dos historias son equivalentes entonces sus grafos de seriabilidad son idénticos.
- Probar o dar un contraejemplo para el recíproco.
- Probar o dar un contraejemplo que si dos historias H1 y H2 son sobre el mismo conjunto de transacciones, tienen las mismas operaciones y el mismo grafo de seriabilidad, entonces son equivalentes.

Ejercicio 5

Sean las siguientes dos transacciones:

T1 : r1(X), w1(X), r1(Y), w1(Y)

T2 : r2(Y), w2(Y), r2(Z), w2(Z)

Dar un ejemplo de ejecuciones concurrentes de estas transacciones que sigan el protocolo 2PL.

Ejercicio 6

a. Dadas las siguientes transacciones:

T1:

leer_elemento(X);

$X := X * N$;

escribir_elemento(X);

leer_elemento(Y);

$Y := Y + N$;

escribir_elemento(Y);

T2:

leer_elemento(X);

$X := X + M$;

escribir_elemento(X);

1. Dar una historia de T1 y T2 recuperable y otra no recuperable. Justificar.

2. Dadas las siguientes historias de T1 y T2:

H1: r1(X), r2(X), w2(X), c2, w1(X), r1(Y), w1(Y), c1

H2: r1(X), w1(X), r1(Y), r2(X), w1(Y), w2(X), c2, c1

i. Decir si son serializables y si son recuperables, justificando las respuestas.

En los

casos en que sea serializable dar la historia serial equivalente.

ii. ¿H1 y H2 son historias equivalentes? ¿Por qué?

b. Dadas

T1: r1(X), r1(Y), w1(Y) y

T2: r2(X), w2(X), r2(X), w2(X)

que bloquean y desbloquean los objetos de la siguiente forma:

T1: rl1(X), r1(X), rl1(Y), r1(Y), wl1(Y), u1(X), w1(Y), u1(Y)

T2: rl2(X), r2(X), wl2(X), w2(X), rl2(X), r2(X), wl2(X), w2(X), u2(X)

siendo: rl = read_lock, wl = write_lock, u = unlock

1. Decir si T1 y T2 siguen el protocolo 2PL. Justificar.

2. Dar una historia que no sea serializable.